Computer Architecture Lab – Branch Prediction and Cache Replacement Study

ChampSim Based Implementation Report

Name: Samyak Bansal
Course: Computer Architecture Laboratory
Simulator: ChampSim
Platform: macOS
Traces Used:

- 401.bzip2-277B.champsimtrace.xz
- 403.gcc-16B.champsimtrace.xz
- 462.libquantum-1343B.champsimtrace.xz
- 605.mcf_s-1644B.champsimtrace.xz

---

1. Overview

This submission implements and evaluates branch prediction and cache replacement mechanisms using the ChampSim simulator.

The work completed includes:

Task 1

- Baseline branch predictor evaluation
- Modified Bullseye branch predictor
- Adaptive Bullseye branch predictor

Task 2

- LLC size sensitivity study
- LLC associativity sensitivity study
- MRU replacement policy
- Random replacement policy
- HC-RRIP replacement policy
- Comparative evaluation of all replacement policies

---

2. Repository Structure

Branch Predictors

Location:

```
branch/
```

Implemented files:

```
branch/bimodal.bpred
branch/gshare.bpred
branch/perceptron.bpred
branch/hashed_perceptron.bpred
branch/modified_bullseye.bpred
branch/adaptive_bullseye.bpred
```

Replacement Policies

Location:

```
replacement/
```

Files:

```
replacement/lru.llc_repl
replacement/srrip.llc_repl
replacement/drrip.llc_repl
replacement/ship.llc_repl
replacement/mru.llc_repl
replacement/random.llc_repl
replacement/hc_rrip.llc_repl
```

Traces

Location:

```
dpc3_traces/
```

Contents:

```
401.bzip2-277B.champsimtrace.xz
403.gcc-16B.champsimtrace.xz
462.libquantum-1343B.champsimtrace.xz
605.mcf_s-1644B.champsimtrace.xz
```

Results

Location:

```
results_10M/
```

Contains simulation outputs for all experiments.

Example:

```
403.gcc-16B.champsimtrace.xz-bimodal-no-no-no-no-lru-1core.txt
```

---

3. Task 1 – Branch Prediction

3.1 Baseline Predictors Evaluated

The following branch predictors were built and executed:

```
Bimodal
GShare
Perceptron
Hashed Perceptron
```

Build Example:

```
./build_champsim.sh bimodal no no no no lru 1
```

Execution Example:

```
./run_champsim.sh \
bimodal-no-no-no-no-lru-1core \
1 \
10 \
403.gcc-16B.champsimtrace.xz
```

3.2 Modified Bullseye Predictor

File:

```
branch/modified_bullseye.bpred
```

Design

The predictor extends GShare by introducing:

Hard Branch Table

Tracks frequently mispredicted branch PCs.

Data structures:

```
unordered_map<uint64_t,int> mispred_count
unordered_set<uint64_t> hard_branches
```

Operation

When:

```
prediction != actual outcome
```

the misprediction counter is incremented.

When:

```
mispred_count >= threshold
```

the branch is classified as a hard branch.

Hard branches receive specialized prediction treatment.

Objective

Focus predictor resources on branches responsible for most mispredictions.

3.3 Adaptive Bullseye Predictor

File:

```
branch/adaptive_bullseye.bpred
```

Additional Components

```
dynamic_threshold
recent_mispreds
branch_counter
```

Operation

Every:

```
10000 branches
```

the predictor evaluates recent MPKI.

If MPKI rises:

```
threshold decreases
```

If MPKI falls:

```
threshold increases
```

This dynamically adjusts the number of hard branches being tracked.

Objective

Improve prediction accuracy while minimizing storage overhead.

---

4. Task 2(a) – LLC Size Sensitivity Study

File Modified:

```
inc/cache.h
```

Parameter:

```
#define LLC_SET
```

Configurations Evaluated

1 MB LLC

```
#define LLC_SET NUM_CPUS*1024
```

---

2 MB LLC

```
#define LLC_SET NUM_CPUS*2048
```

---

4 MB LLC

```
#define LLC_SET NUM_CPUS*4096
```

---

8 MB LLC

```
#define LLC_SET NUM_CPUS*8192
```

Objective

Study impact of LLC capacity on:

```
IPC
```

```
Hit Rate
Miss Rate
Execution Time
```

---

5. Task 2(b) – LLC Associativity Study

File Modified:

```
inc/cache.h
```

Parameter:

```
#define LLC_WAY
```

Configurations

Baseline:

```
#define LLC_WAY 16
```

Modified:

```
#define LLC_WAY 8
```

Objective

Measure effect of reduced associativity on:

```
Conflict Misses
Hit Rate
IPC
Execution Time
```

---

6. Task 2(c) – Replacement Policies

Existing Policies

Files:

```
replacement/lru.llc_repl
replacement/srrip.llc_repl
replacement/drrip.llc_repl
replacement/ship.llc_repl
```

Implemented Policy 1 – MRU

File:

```
replacement/mru.llc_repl
```

Design

Instead of evicting:

```
Least Recently Used
```

MRU evicts:

```
Most Recently Used
```

Victim Selection:

```
if(block[set][way].lru == 0)
    return way;
```

Implemented Policy 2 – Random

File:

```
replacement/random.llc_repl
```

Design

Victim chosen randomly.

Victim Selection:

```
return rand() % LLC_WAY;
```

---

7. Task 2(e) – HC-RRIP

File:

```
replacement/hc_rrip.llc_repl
```

Motivation

A small subset of instruction PCs is responsible for a large fraction of cache misses.

HC-RRIP identifies these PCs and provides cache protection.

Components

Hard-to-Cache Identification Table

Tracks:

```
Hits
Misses
```

per PC.

Difficulty Score

Computed as:

```
Misses / (Hits + 1)
```

Classification

If:

```
Score > Threshold
```

PC becomes:

```
Hard-to-Cache
```

Protection Mechanism

Normal SRRIP:

```
RRPV = 2
```

Hard-to-cache insertion:

```
RRPV = 0
```

This significantly delays eviction.

Objective

Improve:

```
LLC Hit Rate
IPC
Memory Performance
```

for difficult workloads.

---

8. Experimental Matrix

Branch Predictors

```
Bimodal
GShare
Perceptron
Hashed Perceptron
Modified Bullseye
Adaptive Bullseye
```

Total:

```
6 predictors
```

Replacement Policies

```
LRU
SRRIP
DRRIP
SHIP
MRU
Random
HC-RRIP
```

Total:

```
7 policies
```

Traces

```
401.bzip2
403.gcc
462.libquantum
605.mcf
```

Total:

```
4 traces
```

Total Configurations

```
6 × 7 × 4 = 168 runs
```

---

9. Deliverables Checklist

Source Code

- Modified Bullseye Predictor
- Adaptive Bullseye Predictor
- MRU Replacement Policy
- Random Replacement Policy
- HC-RRIP Replacement Policy

Experimental Results

- Branch predictor comparison
- LLC size study
- LLC associativity study
- Replacement policy comparison
- HC-RRIP evaluation

Output Files

Location:

```
results_10M/
```

Contains all simulation logs and performance statistics.

---

10. Build and Run Commands

Example Build:

```
./build_champsim.sh bimodal no no no no lru 1
```

Example Run:

```
./run_champsim.sh \
bimodal-no-no-no-no-lru-1core \
1 \
10 \
403.gcc-16B.champsimtrace.xz
```

All experiments were executed using the above ChampSim workflow and analyzed using the generated result files.